

vuln_lkm.pdf

How the Vuln-LKM loadable kernel module works

Repository: github.com/mayday5000/Vuln-LKM

Device node: `/dev/vuln_lkm`

Module: `vuln_lkm.ko` (misc character driver)

One chapter per function. Lab only — not an exploit guide.

Educational material for the Vuln-LKM loadable kernel module. Not an exploit guide. Integer wrap and a bounded-struct string copy are described as they appear in the source.

Contents

1. What this object is
2. Architecture pictures
3. Shared ABI (vuln_lkm.h)
4. Function: g_val (module state)
5. Function: copy_int_from_user
6. Function: vuln_lkm_add
7. Function: vuln_lkm_sub
8. Function: vuln_lkm_str
9. Function: vuln_lkm_ioctl
10. Data: file_operations and miscdevice
11. Function: vuln_lkm_init
12. Function: vuln_lkm_exit
13. How the pieces call each other
14. Online material
15. Annotated full source
16. Purpose cheat sheet
17. printk lines, one per function
18. Worked packing of VULN_LKM_GET
19. Kernel APIs these functions call
20. How userspace launches these functions

1. What this object is

vuln_lkm is a Linux Loadable Kernel Module (LKM). That sentence has two claims that are easy to blur. First, it is kernel code: after insmod it shares the kernel address space, can call `printk` and `copy_from_user`, and a bug in it can panic the machine. Second, it is loadable: it is not linked into `vmlinux`. You build a `.ko` against the headers of the kernel that will load it, then insert it at runtime. `rmmod` tears it down.

It is also a driver, in the narrow character-device sense. It is not a PCI driver, not a platform driver, not a USB driver, not a block driver. It does not bind to hardware. It registers a `miscdevice` named `vuln_lkm`. The kernel then creates `/dev/vuln_lkm`. Userspace opens that node and talks through `ioctl`. That is the entire I/O surface.

The module exists as a lab. Three operations are intentionally wrong: unsigned 32-bit addition without a saturation check, unsigned 32-bit subtraction without an underflow check, and `memcpy` into a 32-byte kernel stack buffer using a length that is only capped at 256 (the `ioctl` struct), not at 32. GET and SET are the honest controls so you can read and plant `g_val` before ADD or SUB.

Purpose of this PDF

Explain every function in `vuln_lkm.c` and every type in `vuln_lkm.h`: what it is for, who calls it, what it returns, and which lab bug (if any) lives there. Pictures first, then the functions, then links to kernel.org and related reading.

1.1 LKM versus built-in driver

A built-in driver is compiled into `vmlinux`. Its `initcall` runs as the kernel boots. You cannot `rmmod` it. An LKM is a relocatable ELF object. The module loader (`kernel/module.c`) applies relocations, resolves exported symbols (`printk`, `copy_from_user`, `misc_register`), calls the function marked `module_init`, and keeps a struct module around for `sysfs` and `rmmod`.

`kbuild` for an out-of-tree module is the Makefile line `obj-m += vuln_lkm.o` plus `make -C /lib/modules/$(uname -r)/build M=$PWD modules`. The running kernel's build tree supplies `Module.symvers`, compiler flags, and `vermagic`. If `vermagic` does not match, `insmod` refuses.

1.2 Why miscdevice

Linux character drivers historically called `register_chrdev` or `cdev_init` plus `class_create` plus `device_create`. `miscdevice` is a shortcut for one minor under the shared `misc` major (10). `MISC_DYNAMIC_MINOR` asks the core to pick a free minor. The name field becomes the `/dev` node. `mode = 0666` is an explicit lab choice so the CLI can open the node without being root after `insmod`; a production driver would not do that.

The only `file_operations` we fill are `owner` and `unlocked_ioctl`. There is no `.read`, no `.write`, no `.mmap`. `cat` the node and you get the VFS default error. All lab traffic is `ioctl`. `unlocked_ioctl` is the post-2.6.11 path: no Big Kernel Lock. The driver is responsible for its own serialization. `vuln_lkm` has none: `g_val` is a naked static `u32`.

2. Architecture pictures

Four drawings. They are schematic, not a kernel intern's exact call graph. Use them while you read the functions.

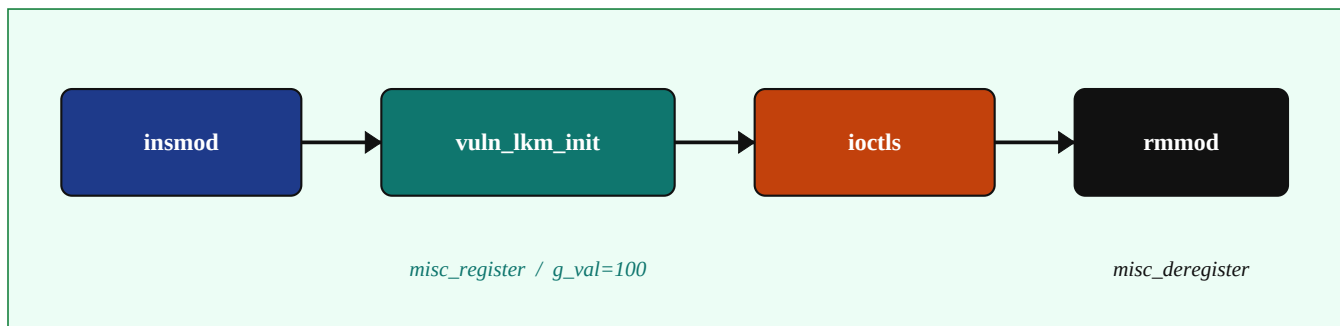


Figure 1. Module lifetime: `insmod` runs `vuln_lkm_init`, `ioctl`s run until `rmmod` runs `vuln_lkm_exit`.

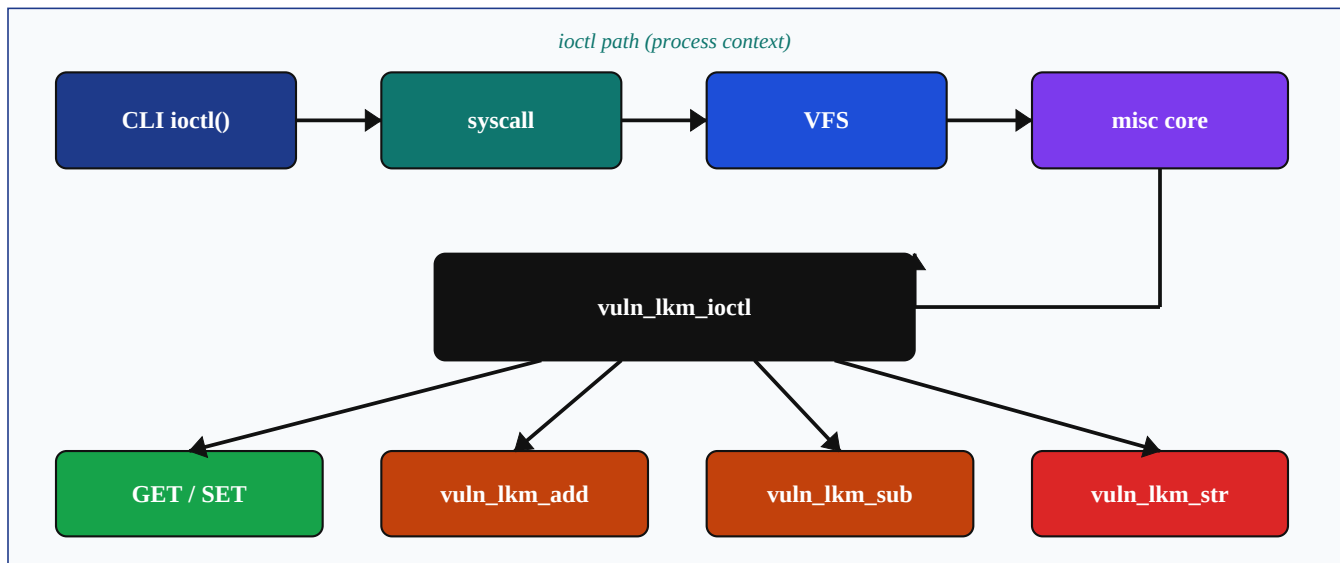


Figure 2. `ioctl` path from the CLI through the `syscall` and `VFS` into `vuln_lkm_ioctl`, then the helpers.

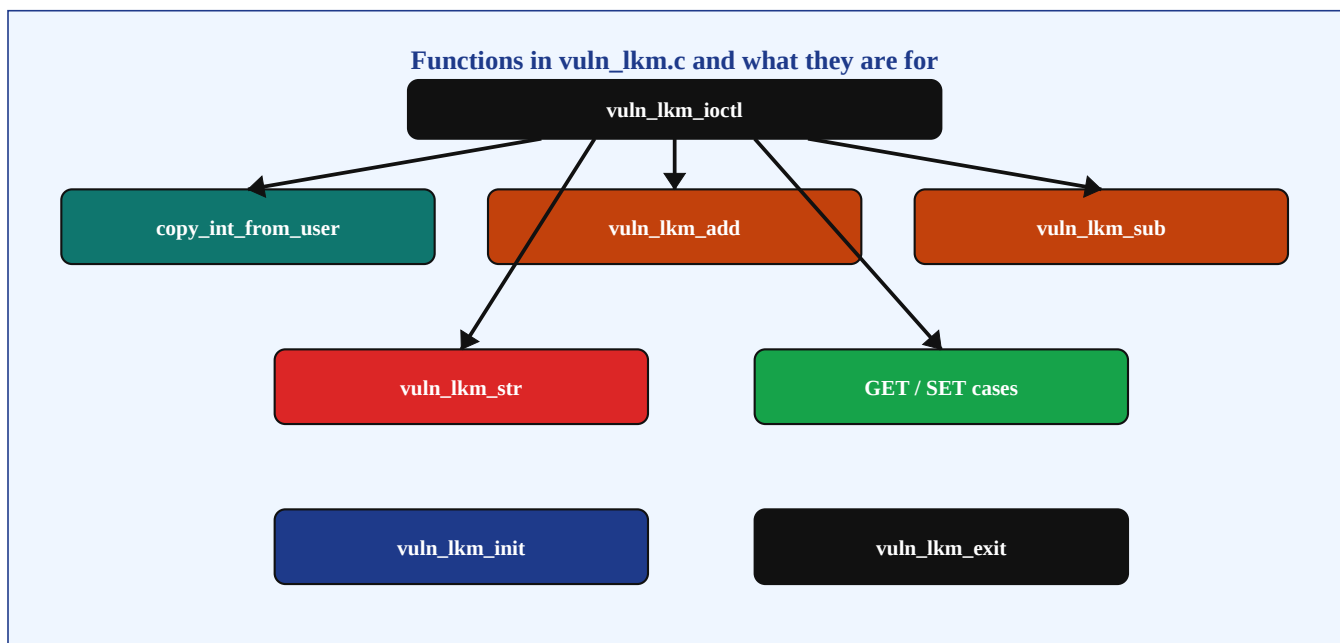


Figure 3. Functions in `vuln_lkm.c`. Orange is wrap. Red is the string smash. Green is the honest GET/SET.

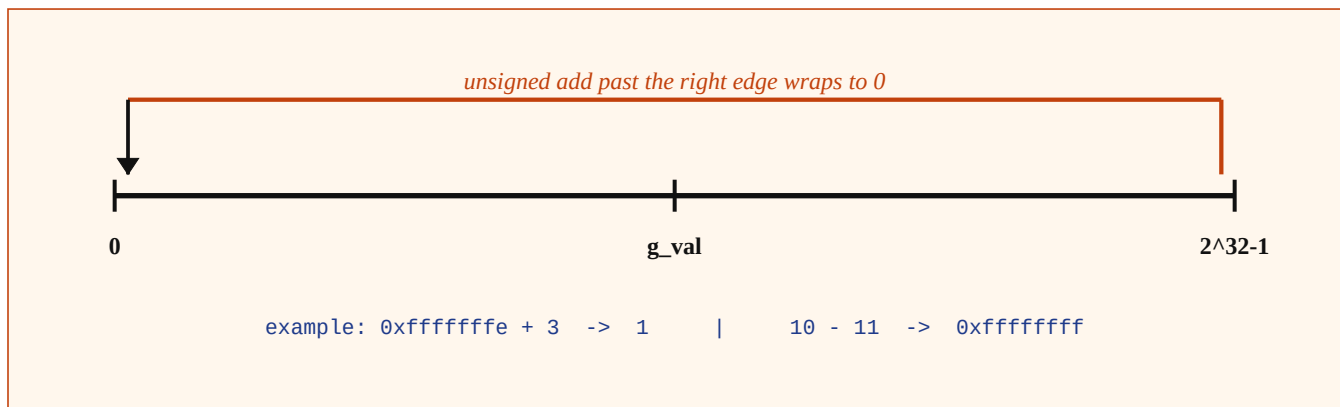


Figure 4. Unsigned u32 wrap: the number line is a ring. ADD past $2^{32}-1$ lands near 0. SUB below 0 lands near $2^{32}-1$.



Figure 5. STR size split. copy_from_user of 260 bytes is fine. memcpy into kbuf[32] with len=40 is not.

3. Shared ABI: vuln_lkm.h

The header is compiled both in the kernel (linux/ioctl.h, linux/types.h) and in the CLI (sys/ioctl.h, stdint.h). That dual include is the UAPI idea: ioctl numbers and struct layouts must match on both sides of copy_from_user. Purpose: one source of truth for magic, sizes, and commands.

```

1  #ifndef VULN_LKM_H
2  #define VULN_LKM_H
3
4  #ifdef __KERNEL__
5  #include <linux/ioctl.h>
6  #include <linux/types.h>
7  #else
8  #include <sys/ioctl.h>
9  #include <stdint.h>
10 typedef uint32_t __u32;
11 #endif
12
13 #define VULN_LKM_NAME "vuln_lkm"
14 #define VULN_LKM_IOC_MAGIC 0x1B
15 #define VULN_LKM_STR_MAX 256
16 #define VULN_LKM_KBUF 32
17
18 struct vuln_lkm_int {
19     __u32 value;
20 };
21

```

```

22 struct vuln_lkm_str {
23     __u32 len;
24     char data[VULN_LKM_STR_MAX];
25 };
26
27 #define VULN_LKM_GET _IOR(VULN_LKM_IOC_MAGIC, 1, struct vuln_lkm_int)
28 #define VULN_LKM_SET _IOW(VULN_LKM_IOC_MAGIC, 2, struct vuln_lkm_int)
29 #define VULN_LKM_ADD _IOW(VULN_LKM_IOC_MAGIC, 3, struct vuln_lkm_int)
30 #define VULN_LKM_SUB _IOW(VULN_LKM_IOC_MAGIC, 4, struct vuln_lkm_int)
31 #define VULN_LKM_STR _IOW(VULN_LKM_IOC_MAGIC, 5, struct vuln_lkm_str)
32
33 #endif

```

What each #define is for

VULN_LKM_NAME is the misc name and the /dev node. VULN_LKM_IOC_MAGIC 0x1B is the ioctl type byte. VULN_LKM_STR_MAX 256 is the ioctl blob. VULN_LKM_KBUF 32 is the kernel stack buffer. Those last two numbers are different on purpose.

3.1 struct vuln_lkm_int

Purpose: carry one u32 across the syscall. GET writes it (kernel to user). SET/ADD/SUB read it (user to kernel). Four bytes, no padding on this ABI. The CLI base64 dump of this struct is eight characters with padding.

3.2 struct vuln_lkm_str

Purpose: carry a counted string: len plus data[256], 260 bytes total. The kernel copies the whole object, then memcpy's req.len bytes into a 32-byte kbuf. The struct is the wire format. kbuf is not.

3.3 ioctl macros

On Linux, _IOC packs nr (bits 0-7), type (8-15), size (16-29), dir (30-31). _IOC_WRITE is 1, _IOC_READ is 2. Typical x86_64 packing:

- VULN_LKM_GET _IOR(0x1B,1,struct vuln_lkm_int) = 0x80041b01 purpose: read g_val
- VULN_LKM_SET _IOW(0x1B,2,struct vuln_lkm_int) = 0x40041b02 purpose: plant g_val
- VULN_LKM_ADD _IOW(0x1B,3,struct vuln_lkm_int) = 0x40041b03 purpose: wrap add
- VULN_LKM_SUB _IOW(0x1B,4,struct vuln_lkm_int) = 0x40041b04 purpose: wrap sub
- VULN_LKM_STR _IOW(0x1B,5,struct vuln_lkm_str) = 0x41041b05 purpose: string copy lab

The kernel switch compares the full 32-bit word. A size mismatch is a different command and returns -ENOTTY.

4. Function: g_val (module state)

g_val is not a function. It is a static u32 at file scope, initialized to 100. Purpose: the only persistent lab state. GET reads it. SET stores it. ADD and SUB mutate it with wrapping arithmetic. STR does not touch it. After rmmmod the storage is gone; the next insmod starts at 100 again.

```

1 static u32 g_val = 100;

```

Purpose

A visible number you can GET after ADD/SUB so wrap is not a mystery. 100 is a better default than 0 (which looks like a failed copy). There is no lock. Two concurrent ioctls can tear it. The CLI is single-threaded in the exercises.

In GDB, after add-symbol-file, p g_val prints this object. It lives in the module's data/bss, not on the ioctl stack. kbuf is the opposite: it exists only while vuln_lkm_str is running.

5. Function: copy_int_from_user

Purpose: the safe copy helper for every integer ioctl that reads from userspace (SET, ADD, SUB). It is not used by GET (GET copies the other way) or STR (STR copies a different struct). It exists so the three cases do not each open-code copy_from_user.

```
1 static int copy_int_from_user(unsigned long arg, struct vuln_lkm_int *out)
2 {
3     if (copy_from_user(out, (void __user *)arg, sizeof(*out)))
4         return -EFAULT;
5     return 0;
6 }
```

5.1 Arguments and return

arg is the ioctl third argument: a userspace pointer. out is a kernel pointer to a local struct vuln_lkm_int. The function copies sizeof(struct vuln_lkm_int) bytes, which is 4. copy_from_user returns the number of bytes not copied. Any leftover becomes -EFAULT. Zero leftover becomes 0.

5.2 Why (void __user *)

__user is a sparse/noderef annotation: this address is not a kernel pointer. The compiler and sparse can warn if you dereference it with ordinary C. copy_from_user is the API that is allowed to touch it. Purpose of the cast: match that API. Purpose of the helper: one place to get the EFAULT policy right.

5.3 Who calls it

vuln_lkm_ioctl, SET/ADD/SUB cases only. GET uses copy_to_user directly. STR uses copy_from_user of sizeof(struct vuln_lkm_str) inside vuln_lkm_str. If you add a new integer command, call this helper, do not dereference arg.

No lab bug here

This helper is the honest copy. The wrap lives in add/sub after a successful copy. The smash lives in memcpy after STR's own copy.

6. Function: vuln_lkm_add

Purpose: demonstrate unsigned 32-bit overflow (modular wrap) on g_val. GDB breakpoint target. Called only from the VULN_LKM_ADD case after copy_int_from_user succeeded.

```
1 static int vuln_lkm_add(u32 n)
2 {
3     u32 before = g_val;
4     /* No check against U32_MAX - n. The add wraps. */
5     g_val = g_val + n;
6     printk(KERN_INFO "vuln_lkm: ADD %u + %u = %u (wrap=%s)\n",
7            before, n, g_val, (g_val < before) ? "yes" : "no");
8     return 0;
9 }
```

6.1 Step by step

Save before so printk can show the old value even after the store. Add n with C unsigned rules: modulo 2^{32} . There is no if ($n > U32_MAX - g_val$) return -EINVAL. printk uses wrap=yes when the result is less than before, which is the usual detector for wrapping addition of a positive n. Always returns 0; wrap is not treated as an error.

6.2 Why this is a lab bug and not UB

Unsigned overflow in C is defined. Signed overflow is undefined. The type is u32 on purpose. The lab lesson is not 'the compiler may delete this'. The lesson is 'this wrapped value would be a wrong size or index if you used it as one'. Here the wrapped value is the artifact: GET it and look.

6.3 Example

SET 0xffffffff, ADD 3. $4294967294 + 3 = 4294967297$ which is $1 \bmod 2^{32}$. dmesg: ADD 4294967294 + 3 = 1 (wrap=yes). GDB: break vuln_lkm_add, p n, p g_val, next across the add, p g_val.

Purpose in one line

Wrap g_val upward with no saturation, log it, return success so the CLI can GET the result.

6.4 What a check would look like (not in the source)

if (check_add_overflow(g_val, n, &g_val)) return -EINVAL; using linux/overflow.h, or the hand form if ($n > U32_MAX - g_val$) return -EINVAL; else $g_val += n$. The committed code does neither. Do not patch this file to 'fix' the lab unless you are done with the exercise.

7. Function: vuln_lkm_sub

Purpose: demonstrate unsigned 32-bit underflow (still modular wrap) on g_val. Mirror of vuln_lkm_add. GDB breakpoint. Called only from the VULN_LKM_SUB case after a successful copy_int_from_user.

```
1 static int vuln_lkm_sub(u32 n)
2 {
3     u32 before = g_val;
4     /* No check against n > g_val. The sub wraps. */
5     g_val = g_val - n;
6     printk(KERN_INFO "vuln_lkm: SUB %u - %u = %u (wrap=%s)\n",
7             before, n, g_val, (g_val > before) ? "yes" : "no");
8     return 0;
9 }
```

7.1 Step by step

Save before. Subtract n with u32 rules. No if ($n > g_val$). printk wrap=yes when the result is greater than before (the subtraction crossed zero). Always returns 0.

7.2 Example

SET 10, SUB 11. $10 - 11$ wraps to 4294967295 (0xffffffff). dmesg: SUB 10 - 11 = 4294967295 (wrap=yes). GET confirms it. GDB: break vuln_lkm_sub, p before, p n, step the sub, p g_val.

Purpose in one line

Wrap `g_val` downward with no underflow check, log it, return success.

7.3 Why add and sub are separate functions

They could be one function with a sign. They are split so GDB can break on one vulnerability at a time, matching scripts/vuln_lkm.gdb. `printk` strings also differ (ADD vs SUB, wrap detector flipped). Purpose of the split: teachable breakpoints, not a software-engineering abstraction.

8. Function: vuln_lkm_str

Purpose: demonstrate a counted copy into a destination that is smaller than the cap. GDB breakpoint. Called only from the VULN_LKM_STR case. This is the string lab. Run lengths above 32 in QEMU.

```

1 static int vuln_lkm_str(unsigned long arg)
2 {
3     struct vuln_lkm_str req;
4     char kbuf[VULN_LKM_KBUF];
5
6     if (copy_from_user(&req, (void __user *)arg, sizeof(req)))
7         return -EFAULT;
8     if (req.len > VULN_LKM_STR_MAX)
9         req.len = VULN_LKM_STR_MAX;
10    memset(kbuf, 0, sizeof(kbuf));
11    printk(KERN_INFO "vuln_lkm: STR len=%u into kbuf[%u]\n",
12           req.len, VULN_LKM_KBUF);
13    memcpy(kbuf, req.data, req.len);
14    return 0;
15 }
```

8.1 Locals and their sizes

`req` is struct `vuln_lkm_str` on the kernel stack: 260 bytes. `kbuf` is `char[32]`. Two objects. The first copy targets `req` and is sized with `sizeof(req)`, which is correct. The second copy targets `kbuf` and is sized with `req.len`, which is not compared to 32.

8.2 Line by line

`copy_from_user` of the whole `ioctl` blob: if the user pointer is bad, `-EFAULT`, `kbuf` untouched. Then if (`req.len > 256`) `req.len = 256`. That cap keeps `memcpy` from reading past `req.data`. It does not keep `memcpy` from writing past `kbuf`. `memset` zeros 32 bytes so a short string looks like a C string if you inspect `kbuf` in GDB. `printk` logs the two sizes on purpose so `dmesg` shows the mismatch (`len=40 into kbuf[32]`). `memcpy` uses `req.len`. return 0 even after a smash.

8.3 What is not here

No `printk` of the bytes (that would be log injection). No NUL-termination of `kbuf` after a 32-byte copy. No `kmalloc`. No `check_add_overflow`. No `copy_from_user` into `kbuf` (the user bytes already live in `req`). The overflow is kernel-to-kernel `memcpy`.

Purpose in one line

Copy a user-chosen number of bytes into a 32-byte stack buffer after a cap that only knows about 256.

8.4 Safe lengths versus unsafe

len 0..32: memcpy is in-bounds. len 5 ('hello') is the control. len 33 and up: overflow. The CLI help uses 40 A's so nobody argues about off-by-one. A 256 cap means the worst memcpy is 256 bytes into 32. That can oops. This PDF does not describe how to aim that write at a return address.

8.5 Layout after kbuf

Compiler-dependent: padding, stack canary (CONFIG_STACKPROTECTOR), saved registers, the rest of req, return address into vuln_lkm_ioctl. GDB: p &kbuf, p &req, p req.len, p sizeof(kbuf). Compare addresses. That is enough to see the objects. Do not build a payload.

9. Function: vuln_lkm_ioctl

Purpose: the unlocked_ioctl implementation. The only entry from the VFS. Signature matches file_operations.unlocked_ioctl: long (*)(struct file *, unsigned int cmd, unsigned long arg). Every CLI command hits this first. GDB breakpoint on the dispatcher.

```

1 static long vuln_lkm_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
2 {
3     struct vuln_lkm_int req = {0};
4
5     switch (cmd) {
6     case VULN_LKM_GET:
7         req.value = g_val;
8         if (copy_to_user((void __user *)arg, &req, sizeof(req)))
9             return -EFAULT;
10        printk(KERN_INFO "vuln_lkm: GET %u (0x%x)\n", g_val, g_val);
11        return 0;
12    case VULN_LKM_SET:
13        if (copy_int_from_user(arg, &req))
14            return -EFAULT;
15        printk(KERN_INFO "vuln_lkm: SET %u -> %u\n", g_val, req.value);
16        g_val = req.value;
17        return 0;
18    case VULN_LKM_ADD:
19        if (copy_int_from_user(arg, &req))
20            return -EFAULT;
21        return vuln_lkm_add(req.value);
22    case VULN_LKM_SUB:
23        if (copy_int_from_user(arg, &req))
24            return -EFAULT;
25        return vuln_lkm_sub(req.value);
26    case VULN_LKM_STR:
27        return vuln_lkm_str(arg);
28    default:
29        return -ENOTTY;
30    }
31 }
```

9.1 Arguments

filp is the struct file from open(/dev/vuln_lkm). Unused: the driver has no per-open state. cmd is the packed ioctl word from the CLI. arg is the userspace pointer to vuln_lkm_int or vuln_lkm_str. Local req is only for the integer commands; STR has its own req inside vuln_lkm_str.

9.2 GET case — purpose

Honest read. Fill req.value from g_val, copy_to_user four bytes, printk, return 0. This is how you prove wrap. No bug. If the user pointer is unmapped, -EFAULT and g_val unchanged.

9.3 SET case — purpose

Honest write. copy_int_from_user, printk old to new, store. Purpose: plant 10 or 0xffffffff before SUB/ADD. A store does not wrap. No bug.

9.4 ADD and SUB cases — purpose

Copy then delegate. The dispatcher does not do arithmetic. Helpers exist for GDB. If the copy fails, the helper is not called and g_val does not change.

9.5 STR case — purpose

No integer copy. Pass arg straight into vuln_lkm_str, which copies the 260-byte struct itself. Returning that function's int as long is fine (0 or -EFAULT).

9.6 default — purpose

-ENOTTY means 'not a command this device knows'. Userspace sees Inappropriate ioctl for device. Happens if the CLI was built against a different header than the .ko.

Purpose in one line

Turn a 32-bit cmd word into GET, SET, add, sub, str, or ENOTTY, with user copies on the integer paths.

9.7 Return type long

unlocked_ioctl returns long because some ioctls return a non-negative length. Negative is -errno. This driver only returns 0 or a negative errno. The VFS puts that in the syscall return.

10. Data: file_operations and miscdevice

These are not functions. They are the objects misc_register and the VFS consult. Purpose: wire unlocked_ioctl to the name /dev/vuln_lkm.

```
1 static const struct file_operations vuln_lkm_fops = {
2     .owner = THIS_MODULE,
3     .unlocked_ioctl = vuln_lkm_ioctl,
4 };
5
6 static struct miscdevice vuln_lkm_dev = {
7     .minor = MISC_DYNAMIC_MINOR,
8     .name = VULN_LKM_NAME,
9     .fops = &vuln_lkm_fops,
10    .mode = 0666,
11 };
```

10.1 vuln_lkm_fops

owner = THIS_MODULE so the module refcount rises while a file is open; rmmmod waits. unlocked_ioctl = vuln_lkm_ioctl is the only method. Missing .read/.write/.mmap/.compat_ioctl/.llseek means those syscalls hit VFS defaults. Purpose: smallest fops that still speaks ioctl.

10.2 vuln_lkm_dev

minor = MISC_DYNAMIC_MINOR: ask misc for a free minor under major 10. name = "vuln_lkm": the /dev node. fops: the table above. mode = 0666: world-accessible in the lab. Purpose: one object to pass to misc_register / misc_deregister.

Not a bus driver

There is no probe, no pci_device_id, no platform_driver. Character-device 'driver' only. See vuln_lkm_run_debug.pdf appendix A if you want the full driver model.

11. Function: vuln_lkm_init

Purpose: module_init. Called once by the loader after relocations. Must return 0 to keep the module, or a negative errno to abort the load (no half-registered device, because we return immediately on misc_register failure).

```
1 static int __init vuln_lkm_init(void)
2 {
3     int ret = misc_register(&vuln_lkm_dev);
4     if (ret)
5         return ret;
6     printk(KERN_INFO "vuln_lkm: loaded, /dev/%s, g_val=%u\n",
7            VULN_LKM_NAME, g_val);
8     return 0;
9 }
```

__init places this function in a section the kernel can free after init on a built-in driver. For an LKM it still marks init. misc_register allocates the minor, creates the class device, and the /dev node (devtmpfs). printk is the 'it worked' banner: device name and g_val=100. Nothing else is allocated. No workqueue, no kthread, no irq.

Purpose in one line

Register /dev/vuln_lkm and announce g_val.

11.1 Failure modes

misc_register can fail if the name collides or the class machinery is unhappy. You then never see /dev/vuln_lkm. load.sh also tries rmmmod of old names (mini_vuln, int_bounds) so a leftover module does not sit on the name. If insmod says Invalid module format, that is vermagic, not this function.

12. Function: vuln_lkm_exit

Purpose: module_exit. Called on rmmmod when the refcount is zero (no open fd). Deregister the misc device so /dev/vuln_lkm vanishes, then printk. No kfree: g_val is static, kbuf was stack. If a CLI still holds the fd, THIS_MODULE's refcount keeps rmmmod from running this until close.

```
1 static void __exit vuln_lkm_exit(void)
2 {
3     misc_deregister(&vuln_lkm_dev);
```

```
4     printk(KERN_INFO "vuln_lkm: unloaded\n");
5 }
```

Purpose in one line

Undo init: drop the char node, log unloaded.

`module_init(vuln_lkm_init)` and `module_exit(vuln_lkm_exit)` are macros that fill struct module's init/exit pointers. `MODULE_LICENSE("GPL")` is required to use GPL-only exports; `printk` and `copy_from_user` are available either way, but many APIs are not. `MODULE_AUTHOR` and `MODULE_DESCRIPTION` are metadata in `modinfo`.

13. How the pieces call each other

`insmod -> vuln_lkm_init -> misc_register`. Userspace `open -> misc open` (not our code) `-> struct file with our fops. ioctl -> VFS -> vuln_lkm_ioctl`. From there:

- GET: `copy_to_user(g_val)` inside the switch. No helper function.
- SET: `copy_int_from_user` then store `g_val`.
- ADD: `copy_int_from_user` then `vuln_lkm_add`.
- SUB: `copy_int_from_user` then `vuln_lkm_sub`.
- STR: `vuln_lkm_str(arg)` which `copy_from_user`'s the 260-byte struct then `memcpy`.
- unknown cmd: `-ENOTTY`.
- `rmmod -> vuln_lkm_exit -> misc_deregister`.

That is the entire dynamic call graph. No timers, no kthreads, no notifier chains. GDB breakpoints on the four names in `scripts/vuln_lkm.gdb` cover the dispatcher and the three bugs.

14. Online material

Read these next to the source. They are the kernel's own docs and the usual textbooks, not exploit writeups.

14.1 Kernel trees and headers

- <https://elixir.bootlin.com/linux/latest/source/include/linux/miscdevice.h> — struct `miscdevice`
- <https://elixir.bootlin.com/linux/latest/source/drivers/char/misc.c> — `misc_register` implementation
- <https://elixir.bootlin.com/linux/latest/source/include/linux/fs.h> — struct `file_operations`
- <https://elixir.bootlin.com/linux/latest/source/include/uapi/asm-generic/ioctl.h> — `_IOC` / `_IOR` / `_IOW` packing
- <https://elixir.bootlin.com/linux/latest/source/include/linux/uaccess.h> — `copy_from_user` / `copy_to_user`
- <https://elixir.bootlin.com/linux/latest/source/include/linux/overflow.h> — `check_add_overflow` (what ADD does not call)
- <https://elixir.bootlin.com/linux/latest/source/kernel/module.c> — loader, `vermagic`, `module_init`

14.2 Documentation in the kernel source

- `Documentation/driver-api/driver-model/` — devices and buses (contrast: we skip this)
- `Documentation/filesystems/vfs.rst` — VFS and `file_operations`
- `Documentation/core-api/kernel-api.rst` — `printk`, memory helpers
- <https://www.kernel.org/doc/html/latest/driver-api/basics.html> — driver API HTML
- <https://www.kernel.org/doc/html/latest/core-api/printk-basics.html> — `printk` levels

14.3 Books and long-form

- Linux Device Drivers, 3rd edition (Corbet, Rubini, Kroah-Hartman) — char drivers; old on APIs, right on ideas. Free from lwn.net/kernel/ldd3/
- Linux Kernel Development (Love) — modules, syscalls, concurrency
- LWN.net kernel index: <https://lwn.net/Kernel/Index/> — unlocked_ioctl history, module loading
- man 2 ioctl, man 2 init_module / finit_module, man 4 misc

14.4 This repo's other PDFs

vuln_lkm_cli.pdf is the userspace tester (every helper in the CLI). vuln_lkm_run_debug.pdf is install, QEMU, GDB, IRQs versus Linux process context, and overflow theory. This file is only the .ko: functions and purpose.

What not to search for

This lab is not a privesc kit. Skip SMEP/KASLR bypass writeups and HEVD exploit trees if your goal is to understand these functions.

15. Annotated full source: vuln_lkm.c

The file as committed, with syntax coloring. GDB breakpoints: vuln_lkm_ioctl, vuln_lkm_add, vuln_lkm_sub, vuln_lkm_str.

```

1 #include <linux/module.h>
2 #include <linux/miscdevice.h>
3 #include <linux/uaccess.h>
4
5 #include "vuln_lkm.h"
6
7 static u32 g_val = 100;
8
9 static int copy_int_from_user(unsigned long arg, struct vuln_lkm_int *out)
10 {
11     if (copy_from_user(out, (void __user *)arg, sizeof(*out)))
12         return -EFAULT;
13     return 0;
14 }
15
16 /* GDB: break vuln_lkm_add */
17 static int vuln_lkm_add(u32 n)
18 {
19     u32 before = g_val;
20
21     /* No check against U32_MAX - n. The add wraps. */
22     g_val = g_val + n;
23     printk(KERN_INFO "vuln_lkm: ADD %u + %u = %u (wrap=%s)\n",
24            before, n, g_val, (g_val < before) ? "yes" : "no");
25     return 0;
26 }
27
28 /* GDB: break vuln_lkm_sub */
29 static int vuln_lkm_sub(u32 n)
30 {
31     u32 before = g_val;
32

```

```

33  /* No check against n > g_val. The sub wraps. */
34  g_val = g_val - n;
35  printk(KERN_INFO "vuln_lkm: SUB %u - %u = %u (wrap=%s)\n",
36          before, n, g_val, (g_val > before) ? "yes" : "no");
37  return 0;
38 }
39
40 /* GDB: break vuln_lkm_str */
41 static int vuln_lkm_str(unsigned long arg)
42 {
43     struct vuln_lkm_str req;
44     char kbuf[VULN_LKM_KBUF];
45
46     if (copy_from_user(&req, (void __user *)arg, sizeof(req)))
47         return -EFAULT;
48
49     if (req.len > VULN_LKM_STR_MAX)
50         req.len = VULN_LKM_STR_MAX;
51
52     memset(kbuf, 0, sizeof(kbuf));
53     printk(KERN_INFO "vuln_lkm: STR len=%u into kbuf[%u]\n",
54            req.len, VULN_LKM_KBUF);
55     /*
56      * kbuf is 32 bytes. req.len is not compared to VULN_LKM_KBUF.
57      * memcpy uses the user-supplied length as-is.
58      */
59     memcpy(kbuf, req.data, req.len);
60     return 0;
61 }
62
63 static long vuln_lkm_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
64 {
65     struct vuln_lkm_int req = {0};
66
67     switch (cmd) {
68     case VULN_LKM_GET:
69         req.value = g_val;
70         if (copy_to_user((void __user *)arg, &req, sizeof(req)))
71             return -EFAULT;
72         printk(KERN_INFO "vuln_lkm: GET %u (0x%x)\n", g_val, g_val);
73         return 0;
74
75     case VULN_LKM_SET:
76         if (copy_int_from_user(arg, &req))
77             return -EFAULT;
78         printk(KERN_INFO "vuln_lkm: SET %u -> %u\n", g_val, req.value);
79         g_val = req.value;
80         return 0;
81
82     case VULN_LKM_ADD:
83         if (copy_int_from_user(arg, &req))
84             return -EFAULT;
85         return vuln_lkm_add(req.value);
86
87     case VULN_LKM_SUB:
88         if (copy_int_from_user(arg, &req))
89             return -EFAULT;
90         return vuln_lkm_sub(req.value);
91
92     case VULN_LKM_STR:

```

```
93     return vuln_lkm_str(arg);
94
95     default:
96         return -ENOTTY;
97     }
98 }
99
100 static const struct file_operations vuln_lkm_fops = {
101     .owner = THIS_MODULE,
102     .unlocked_ioctl = vuln_lkm_ioctl,
103 };
104
105 static struct miscdevice vuln_lkm_dev = {
106     .minor = MISC_DYNAMIC_MINOR,
107     .name = VULN_LKM_NAME,
108     .fops = &vuln_lkm_fops,
109     .mode = 0666,
110 };
111
112 static int __init vuln_lkm_init(void)
113 {
114     int ret = misc_register(&vuln_lkm_dev);
115
116     if (ret)
117         return ret;
118     printk(KERN_INFO "vuln_lkm: loaded, /dev/%s, g_val=%u\n",
119            VULN_LKM_NAME, g_val);
120     return 0;
121 }
122
123 static void __exit vuln_lkm_exit(void)
124 {
125     misc_deregister(&vuln_lkm_dev);
126     printk(KERN_INFO "vuln_lkm: unloaded\n");
127 }
128
129 module_init(vuln_lkm_init);
130 module_exit(vuln_lkm_exit);
131 MODULE_LICENSE("GPL");
132 MODULE_AUTHOR("Damian");
133 MODULE_DESCRIPTION("Educational LKM: integer wrap and string buffer overflow");
```

15.1 Header again

```

1 #ifndef VULN_LKM_H
2 #define VULN_LKM_H
3
4 #ifdef __KERNEL__
5 #include <linux/ioctl.h>
6 #include <linux/types.h>
7 #else
8 #include <sys/ioctl.h>
9 #include <stdint.h>
10 typedef uint32_t __u32;
11 #endif
12
13 #define VULN_LKM_NAME "vuln_lkm"
14 #define VULN_LKM_IOC_MAGIC 0x1B
15 #define VULN_LKM_STR_MAX 256
16 #define VULN_LKM_KBUF 32
17
18 struct vuln_lkm_int {
19     __u32 value;
20 };
21
22 struct vuln_lkm_str {
23     __u32 len;
24     char data[VULN_LKM_STR_MAX];
25 };
26
27 #define VULN_LKM_GET _IOR(VULN_LKM_IOC_MAGIC, 1, struct vuln_lkm_int)
28 #define VULN_LKM_SET _IOW(VULN_LKM_IOC_MAGIC, 2, struct vuln_lkm_int)
29 #define VULN_LKM_ADD _IOW(VULN_LKM_IOC_MAGIC, 3, struct vuln_lkm_int)
30 #define VULN_LKM_SUB _IOW(VULN_LKM_IOC_MAGIC, 4, struct vuln_lkm_int)
31 #define VULN_LKM_STR _IOW(VULN_LKM_IOC_MAGIC, 5, struct vuln_lkm_str)
32
33 #endif

```

16. Purpose cheat sheet

- g_val — persistent u32 lab state, starts at 100
- copy_int_from_user — copy 4 bytes from user for SET/ADD/SUB; returns 0 or -EFAULT
- vuln_lkm_add — g_val += n with u32 wrap; printk wrap=yes; always 0
- vuln_lkm_sub — g_val -= n with u32 wrap; printk wrap=yes; always 0
- vuln_lkm_str — copy 260-byte ioctl struct, cap len at 256, memcpy into kbuf[32]
- vuln_lkm_ioctl — switch on cmd; GET/SET here; ADD/SUB/STR delegate
- vuln_lkm_fops — owner + unlocked_ioctl only
- vuln_lkm_dev — miscdevice name vuln_lkm, dynamic minor, mode 0666
- vuln_lkm_init — misc_register, printk loaded
- vuln_lkm_exit — misc_deregister, printk unloaded

Remember the two sizes

256 is the ioctl string. 32 is the kernel buffer. ADD/SUB wrap a u32. GET/SET are honest. QEMU for STR > 32.

Companion: vuln_lkm_cli.pdf (how the tester sends these ioctls) and vuln_lkm_run_debug.pdf (how to install, step in GDB, and the theory behind ioctl, IRQs, and overflows).

17. printk lines, one per function

Every function that talks to the log has a fixed format. Purpose: you can grep dmesg and know which function ran without GDB.

```
1 init: vuln_lkm: loaded, /dev/vuln_lkm, g_val=100 /* vuln_lkm_init */
2 exit: vuln_lkm: unloaded /* vuln_lkm_exit */
3 GET: vuln_lkm: GET %u (0x%x) /* ioctl GET case */
4 SET: vuln_lkm: SET %u -> %u /* ioctl SET case */
5 ADD: vuln_lkm: ADD %u + %u = %u (wrap=%s) /* vuln_lkm_add */
6 SUB: vuln_lkm: SUB %u - %u = %u (wrap=%s) /* vuln_lkm_sub */
7 STR: vuln_lkm: STR len=%u into kbuf[%u] /* vuln_lkm_str */
8 copy_int_from_user: (no printk; failure is -EFAULT to ioctl)
```

copy_int_from_user is silent on purpose. A failed copy is errno in the CLI, not a log line. STR does not print the bytes. ADD/SUB print wrap=yes/no so you do not have to GET to know wrap happened, but GET is still the ABI-visible proof.

17.1 Mapping a dmesg line back to a source line

If you see wrap=yes on ADD, you are in vuln_lkm_add after g_val = g_val + n. If you see len=40 into kbuf[32], you are in vuln_lkm_str after the cap and before or at memcpy depending on whether printk ran (it runs before memcpy, so a panic on memcpy still leaves that log line). If you never see loaded, init failed or you are on the wrong kernel log.

18. Worked packing of VULN_LKM_GET

Purpose of this chapter: make the 0x80041b01 in the CLI dump stop being magic. _IOR(0x1B, 1, struct vuln_lkm_int).

- dir = _IOC_READ = 2. Bits 31-30 = 10 binary = 0x80000000
- size = 4. Bits 29-16 = 4 = 0x00040000
- type = 0x1B. Bits 15-8 = 0x00001b00
- nr = 1. Bits 7-0 = 0x00000001
- OR them: 0x80041b01

STR is the same exercise with size 260 = 0x104, dir write (0x40000000), nr 5: 0x41041b05. If you change VULN_LKM_STR_MAX, the size field changes, the command word changes, and an old CLI gets ENOTTY from the default branch of vuln_lkm_ioctl. That is why the header is shared and why copy_int_from_user is not used for STR (wrong sizeof).

18.1 Why GET is _IOR and SET is _IOW

Direction is documented from userspace's point of view. _IOR: user reads (kernel writes the buffer). _IOW: user writes (kernel reads the buffer). GET copy_to_user is the _IOR. SET/ADD/SUB/STR copy_from_user are _IOW. The kernel does not auto-copy from the dir bits in unlocked_ioctl; we still call copy_*_user by hand. The bits are the ABI contract and what strace prints.

18.2 Function responsibilities versus the macros

The macros do not call functions. The compiler turns them into integers. vuln_lkm_ioctl compares those integers. The CLI includes the same header so it produces the same integers. Purpose of the header functions? There are none. Purpose of the C functions is everything after the match.

19. Kernel APIs these functions call

Purpose of this chapter: the functions above are thin. The real work is in kernel helpers. Each bullet is 'who calls it in this module' plus 'what it is for'.

- `copy_from_user` — `copy_int_from_user` and `vuln_lkm_str`. Copy user memory into a kernel object. Returns leftover bytes. May sleep.
- `copy_to_user` — GET case in `vuln_lkm_ioctl`. Opposite direction. May sleep.
- `memcpy` — `vuln_lkm_str` only. Kernel-to-kernel, no fault handling, trusts `n`.
- `memset` — `vuln_lkm_str`. Zero `kbuf[32]` before the copy so GDB sees a clean buffer.
- `printk` — `init`, `exit`, `GET`, `SET`, `add`, `sub`, `str`. `KERN_INFO` into the ring buffer and consoles.
- `misc_register` — `vuln_lkm_init`. Attach `vuln_lkm_dev` to `misc` major 10.
- `misc_deregister` — `vuln_lkm_exit`. Undo that.
- `module_init` / `module_exit` — macros that register `init/exit` with struct `module`.
- `MODULE_LICENSE`, `AUTHOR`, `DESCRIPTION` — `modinfo` metadata; `LICENSE GPL` for GPL exports.

19.1 APIs not called, on purpose

No `mutex_lock`, no `spin_lock`, no `kmalloc`, no `copy_from_user` into `kbuf`, no `check_add_overflow`, no `capable()`, no `compat_ioctl`, no `.read`. The lab is small so each function has one job you can break on.

20. How userspace launches these functions

The C functions never run until someone loads the `.ko` and then opens the node. `load.sh` and the `Makefile` are not kernel code, but they are how `vuln_lkm_init` and then `vuln_lkm_ioctl` get CPU time.

20.1 Makefile

`obj-m += vuln_lkm.o` asks `kbuild` to produce `vuln_lkm.ko` from `vuln_lkm.c`. `KDIR` is the build tree of the kernel that will load it. `make user` builds the CLI.

```
1 obj-m += vuln_lkm.o
2 KDIR ?= /lib/modules/$(shell uname -r)/build
3
4 .PHONY: all module user clean
5
6 all: module user
7
8 module:
9     $(MAKE) -C $(KDIR) M=$(CURDIR) modules
10
11 user: vuln_lkm_cli
12
13 vuln_lkm_cli: vuln_lkm_cli.c vuln_lkm.h
14     gcc -Wall -O2 -o $@ vuln_lkm_cli.c
15
16 clean:
17     $(MAKE) -C $(KDIR) M=$(CURDIR) clean
18     rm -f vuln_lkm_cli mini_vuln_cli int_bounds_cli
```

20.2 `load.sh` calls `vuln_lkm_init`

`make module`, `rmmmod` leftover names, `insmod ./vuln_lkm.ko` (that is `init`), `chmod 666` so the CLI can open. If `init misc_register` failed, `insmod` fails and you never `chmod`.

```
1 #!/bin/sh
2 set -e
3 make module
4 sudo rmmod vuln_lkm 2>/dev/null || true
5 sudo rmmod mini_vuln 2>/dev/null || true
6 sudo rmmod int_bounds 2>/dev/null || true
7 sudo insmod ./vuln_lkm.ko
8 sudo chmod 666 /dev/vuln_lkm 2>/dev/null || true
9 echo "loaded /dev/vuln_lkm"
10 ls -l /dev/vuln_lkm
```

20.3 unload.sh calls vuln_lkm_exit

rmmod vuln_lkm runs exit if the refcount is zero. An open CLI fd blocks that because fops.owner is THIS_MODULE.

```
1 #!/bin/sh
2 set -e
3 sudo rmmod vuln_lkm
4 echo "unloaded vuln_lkm"
```

After load, dmesg must show vuln_lkm_init printk. After a GET, dmesg must show the GET line from vuln_lkm_ioctl. That is the practical test that the functions in this PDF are the ones running.

End of the driver manual

You should be able to point at any identifier in vuln_lkm.c and say what it is for. If you cannot, the cheat sheet in chapter 16 is the index. Then open vuln_lkm_cli.pdf for the other side of the ioctl.